Coding conventions

Conventions for writing SWAT+ Fortran source. The goal is portable, readable and reliable code that compiles cleanly on every supported compiler and operating system.

Each item is tagged with one of three levels:

- Required: mandatory. Pull requests that violate these will not be merged.
- Recommended: strongly preferred. Deviations need justification.
- Encouraged: optional. Apply for consistency where reasonable.

The list below is ordered by decreasing impact on code quality. Most subjective style items (indent width, capitalization) are in the lower tiers.

Interoperability and portability

Required

- Source must conform to the ISO Fortran 95 standard.
- No compiler-specific or platform-specific extensions.
- No use of compiler-dependent error specifier values (`IOSTAT`, `STAT`).
- The source must compile with `gfortran` from GCC, even if the production build uses the Intel compiler.
- `EXIT(N)` is the F90/95 standard exit. Prefer it to `STOP`. `STOP` does not always return an error code. If you need an error code passed back to a script, use `EXIT`, and keep its call sites in one central place.

Recommended

- For floating-point precision and integer size, use the F90/95 `KIND` feature. Do not depend on a vendor-supplied default precision flag.
- Do not use tab characters. Tabs are not part of the Fortran character set and break portability.

Encouraged

- For code that interacts with external frameworks, declare every variable with a `KIND` type to make integration easier.

Readability

Required

- Free-form source.
- Consistent indentation, at least two spaces per level.
- Organize source into modules.
- Do not use Fortran keywords (`DATA`, etc.) as variable names.
- Use meaningful names for variables and parameters. Recognized abbreviations are fine when full names would be unwieldy.
- Every externally-called function or subroutine has a header describing what it does, its arguments, and the author. Small internal routines can use a few descriptive comments instead.
- No magic numbers. Physical constants (pi, gas constants) go in `PARAMETER` statements, not hard-coded into expressions.
- Do not hard-code numeric literals in argument lists. The compiler's default precision for literals is not guaranteed.

- No `GOTO`. If you must use one, document why.

Recommended

- Name your loops and other constructs (subroutines, functions, modules, interfaces) where the name aids readability, especially in nested loops.
- Comment input, output, and local variables. Grouping comments across similar variables is fine when their names speak for themselves.
- Comment to mark major functional sections.
- Do not use Fortran statement names or intrinsic function names as symbols.
- Prefer named parameters over numeric literals: `REAL, PARAMETER :: PI = 3.14159, ONE = 1.0`.

Encouraged

- In new code, follow a consistent style. When editing existing code, match the style around you.
- Indent comments the same way as code.
- Group related procedures and data into modules.
- Keep lines to 132 characters or fewer.
- Use `<, >, <=, >=, ==, /=` instead of `.lt., .gt., .le., .ge., .eq., .ne.`.
- Name modules the same as the file that contains them. Avoid putting multiple modules in one file.
- Use blanks around operators, around `=`, and in declarations to separate syntax elements.
- Always use the `::` notation in declarations, even when there are no attributes.
- Line up attributes, variable names, and trailing comments vertically in declarations.
- Remove unused variables.
- Remove debug code once the bug is fixed.

Reliability

Required

- `IMPLICIT NONE` in every program, module, and procedure.
- Use `PRIVATE` in modules by default, then explicitly mark what should be `PUBLIC`. Exception: modules that exist only to expose public data (constants, for example) can stay fully public.
- Initialize every variable. Do not rely on the machine's default.
- Do not initialize a variable of one type with a value of another type.

Recommended

- Do not use `==` or `/=` on floating-point values. Compare against an explicit tolerance instead (an epsilon check).
- In mixed-type expressions and assignments, write the type conversion explicitly. Do not let the compiler do it implicitly.
- No `INCLUDE` files. Use `USE` with modules.
- Derived types belong in a separate module, together with the procedures that operate on them.
- Each procedure should do one thing.
- Module-level (global) public variables should contain only static or rarely-changing data.

Encouraged

- Always use parentheses to make expression evaluation order explicit.
- Prefer derived types (`%` field access) over loose collections of arrays.

Arrays

Required

- Subscript expressions must be integer.
- Do not assume any particular array passing mechanism when passing arrays as arguments.

Recommended

- Use array operations and intrinsics where they apply.
- Pass arrays as assumed shape.

Encouraged

- Declare `DIMENSION` on every non-scalar.

Dynamic memory and pointers

Required

- Prefer `ALLOCATABLE` arrays over pointers where possible. Reduces leak and fragmentation risk.
- Pointers are allowed when a subroutine needs to return a declared array to its caller.
- Initialize every pointer to `NULL()` in its declaration: `INTEGER, POINTER :: x => NULL()`.
- Prefer automatic arrays for transient allocations. `ALLOCATABLE` and `POINTER` arrays need explicit `DEALLOCATE`.

Recommended

- Always deallocate. Especially inside subroutines and loops.
- Check the `STAT` argument on `ALLOCATE` and `DEALLOCATE`.
- Do not repeatedly `ALLOCATE`, `DEALLOCATE`, then `ALLOCATE` a larger block. Heap fragmentation follows.

Encouraged

- Dynamic allocation is preferred over fixed-size declarations with worst-case dimensions.
- Use automatic arrays in subroutines for simplicity.

Looping

Required

- No `GOTO` for loop exit or continuation. Use `EXIT` or `CYCLE`.

Recommended

- No numbered `DO` loops (`DO 10 ... 10 CONTINUE`).

Functions and procedures

Required

- Avoid `SAVE`. Keep state in module variables instead.
- No `ENTRY` statements in functions.
- Functions must not return pointers.
- Do not name a user function the same as an intrinsic (`SUM`, etc.).
- A procedure that returns a single value should be a function. The value can be a derived type.
- Communicate through arguments or through the variables of the procedure's module.

Recommended

- Every dummy argument that is not a pointer has an `INTENT`.
- Avoid type-specific intrinsics (`AMAX`, `DMAX`). Use the generic (`MAX`).
- Do not declare static-dimensioned array arguments.
- Validate argument values where it matters.

Encouraged

- On an error, print a message that includes the routine name. You may terminate inside the routine, or pass an error flag back through the argument list.
- Use overloading when the same operation works on multiple types.
- Use modules or `CONTAINS` for explicit interfaces.
- Avoid external routines. They need separate interface blocks that drift out of sync.

I/O

Required

- External-file I/O statements include `err=`, `end=`, and `iostat=` as appropriate.
- All global variables, if any, are initialized at startup.

Recommended

- Avoid `NAMELIST` I/O.
- Use `WRITE` instead of `PRINT` for non-terminal I/O.
- Use character parameters or in-line format specifiers in `READ` and `WRITE`. Do not use labeled `FORMAT` statements.

Obsolescent features to avoid

Required

- No common blocks. Use modules.
- No assigned or computed `GO TO`. Use `CASE`.
- No arithmetic `IF`. Use a block `IF`.
- Use `REAL`, not `DOUBLE PRECISION`.
- Avoid `DATA`, `ASSIGN`, labeled `DO`, `BACKSPACE`, blank `COMMON`, `BLOCK DATA`.
- Do not branch to an `END IF` from outside its `IF` block.
- No non-integer `DO` loop counters.
- No Hollerith constants.
- No `PAUSE`.
- No multiple `RETURN`, no alternate `RETURN`.

Recommended

- No `EQUIVALENCE`. Use pointers or derived types.

Encouraged

- Do not implicitly reshape arrays when passing them. This was common in F77 and is forbidden by F90 but still possible through external routines without interface blocks.

Source files

Required

- Document every function interface: argument name, type, units, description, constraints, defaults.
- No `INCLUDE`. Use `USE`.
- Keep lines (including comments) under 80 columns where the language allows it.

Recommended

- Keep individual procedures under 300-500 effective lines.
- Separate statement blocks with blank lines or a marker character in column 1.
- Indent consistently.
- Module and subprogram names are lower case. The file is named after the contained module or subprogram, with `.f90`.

Encouraged

- Separate argument declarations from local declarations visually.
- Use descriptive unique names. Keep them under 12-15 characters.
- Indent continuation lines so a multi-line expression lines up readably.
- Comments start with a marker character (`!`). A standalone comment line starts the marker in column 1.

References

- SAS Coding Conventions
- WRF Coding Conventions
- USGS Coding Standards

  Sources

  - `doc/coding_conventions.md` in the source repository.